Relatório - Trabalho 1 Algoritmos e Programação II

Marcos Henrique de Souza Almeida
RGA: 202419040110

Este trabalho tem por objetivo aglutinar alguns métodos de ordenação estudados em sala de aula, e verificar as diferenças de tempo em cada um deles:

- **Merge-Sort**: Algoritmo de divisão e conquista de complexidade $O(N \log N)$. Divide o vetor recursivamente ao meio até obter subvetores unitários e depois os intercala de forma ordenada (no caso, decrescente) utilizando um vetor auxiliar temporário.

- **Quick-Sort Tradicional**: Algoritmo de divisão e conquista $O(N \log N)$ em média. Funciona escolhendo um elemento como pivô (no meu código, o último elemento do subvetor) e particionando os dados de modo que os elementos maiores fiquem à esquerda e os menores à direita do pivô.

- **Quick-Sort Híbrido (com Insertion-Sort)**: Variação que substitui a recursão pelo Insertion-Sort $O(N^2)$ quando o tamanho do subvetor fica igual ou menor a 15 elementos. Como o Insertion-Sort tem baixíssimo custo para vetores pequenos, essa estratégia reduz o custo de chamadas recursivas na pilha.

- **`qsort()` da biblioteca padrão (`stdlib.h`)**: Função nativa da linguagem C otimizada no nível do sistema e compilador, que serve como parâmetro de referência de desempenho para a comparação.

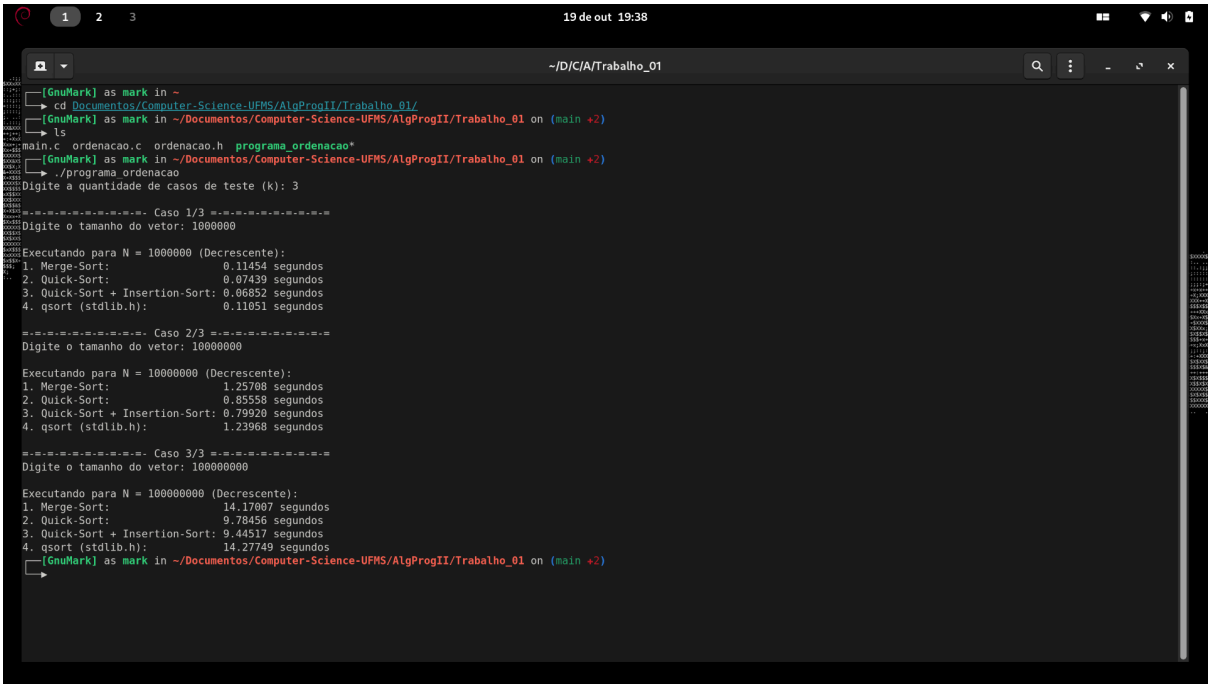
Para isto, o ambiente de testes foi um notebook ThinkPad x395 com as seguintes configurações:

CPU: AMD Ryzen 5 PRO 3500U;
Memória RAM: 16 GB DDR4;
Sistema Operacional: Debian GNU/Linux 13 (trixie) x86_64
gcc: gcc (Debain 14.2.0-19) 14.2.0 (usado flag -O2 para otimização).

Sendo assim, os testes foram feitos diretamente via terminal, a qual é mostrado no print da tela abaixo. Decidi fazer apenas os 3 testes base de 10^6, 10^7 e 10^8:

Método de Ordenação	$N = 10^6$ (1 Milhão)	$N = 10^7$ (10 Milhões)	$N = 10^8$ (100 Milhões)
Merge-Sort	0,11454 s	1,25708 s	14,17007 s
Quick-Sort Tradicional	0,07439 s	0,85558 s	9,78456 s
Quick-Sort + Insertion-Sort	0,06852 s	0,79920 s	9,44517 s

`qsort()` (`stdlib.h`)	0,11051 s	1,23968 s	14,27749 s



Como podemos ver nos resultados, a interrupção na recursão de subvetores pequenos reduz o overhead de chamadas de função na pilha, gerando um ganho substancial de perfomance no Quick-sort + Insertion-sort.

Além disso, o Merge-Sort realiza alocações dinâmicas para intercalar os vetores, exigindo memória extra, enquanto o Quick-Sort ordena no próprio vetor. Ou seja, o Merge-Sort exige o dobro de memória (como podemos ver nos resultados).

Quanto ao qsort(), podemos verificar que ele ficou mais próximo do Merge, apesar de utilizar técnicas internas avanças de seleção e vetorização. Isso mostra que que meu ambiente de compilação muito provavelmente usa a alocação de memória auxiliar, ou verifica de forma genérica por ponteiro de função, adicionando mais um custo na comparação.

É importante frisar também que, a função rand() foi usada para preencher o vetor original de uma única vez por caso de teste, e a função copiar_vetor garantiu que os algoritmos fossem testados com exatamente a mesma sequência desordenada de números. O que garantiu em todo o processo, uma isonomia de entrada.